ChatGPT

Hades AI Service Orchestration: Backend Integration Handoff

Executive Summary: The Hades AI Service orchestrator handles end-to-end learning-path planning and resource discovery. Given a learner's **target goal**, it generates a topic roadmap and fetches learning resources (videos, articles) for each topic. It exposes a single HTTP API endpoint (`POST /ai/orchestrate`) that the Scala backend calls on session start or topic completion. Each call returns the updated session state, including the **active topic**, **active resources**, **prefetched next topics**, and other metadata. This document describes the integration contract and usage patterns so the Scala backend developer can implement and test the interface. It covers startup steps (ensuring both the AI service and database are running and networked), endpoint specification, example requests/responses, event flow, backend responsibilities, error handling, and testing steps.

Startup: Running the AI Service and Database

- **PostgreSQL + pgvector:** Ensure a Postgres container with the `pgvector` extension is running before starting the AI service. For example:

```
docker run -d \
  --name hcl-postgres \
  -e POSTGRES_DB=hcl_learning \
  -e POSTGRES_USER=hcl \
  -e POSTGRES_PASSWORD=hcl_dev_password \
  -p 5432:5432 \
  pgvector/pgvector:pg16
```

This creates a database `hcl_learning` with user `hcl` and password `hcl_dev_password`.

- **Docker Network:** Create a Docker network so both containers can communicate. For example:

```
docker network create hades-net
```

(This is like **Step 1** in the TaskMaster example【4†L180-L184】.) Then run both containers on this network. For the AI service container, use `--network hades-net` (or `docker network connect hades-net hades-ai-service`) so it shares the network with Postgres【13†L232-L240】【11†L108-L113】. This ensures the AI service can resolve the Postgres container by name.

- **Environment Variables:** The AI service reads the database URL from `DATABASE_URL`. **Important:** Do **not** use `localhost` in this URL. Instead, use the Postgres container's name (e.g. `hcl-postgres`) as the host【11†L108-L113】. For example:

```
DATABASE_URL=postgresql://hcl:hcl_dev_password@hcl-postgres:5432/
hcl_learning
```

(This mirrors the correct setup in the Docker Compose snippet【13†L232-L240】.) Make sure your `.env` file is **never committed to version control**; include only a template (`.env.example`) with placeholder values【19†L265-L273】. After updating `.env`, restart the AI container so it picks up the new variables (no need to rebuild the image).

- **AI Service Container:** Build the Docker image for the AI service (if not already done) and run it on the same network. For example:

```
docker build -t hades-ai-service .
docker run --rm \
  --name hades-ai-service \
  --network hades-net \
  --env-file ./.env \
  -p 8000:8000 \
  hades-ai-service
```

Check the logs for `INFO:     Uvicorn running on http://0.0.0.0:8000` to confirm it started correctly.

- **Database Connectivity:** If you encounter errors like *"Connection refused on port 5432"* or *"could not translate host name"*, verify that the database is healthy and that both containers share the network【17†L1040-L1044】. For example, run `docker exec -it hcl-postgres pg_isready -U hcl` and ensure it returns "accepting connections". Use the container name `hcl-postgres` in your URL【11†L108-L113】 (not `localhost` or `127.0.0.1`), and ensure the AI container is on `hades-net` so it can resolve that name.

Endpoint Specification (`POST /ai/orchestrate`)

The AI service listens on `POST /ai/orchestrate`. The request body must be JSON with these fields:

- `learner_id` (string): Learner identifier (e.g. `"psychology-demo-001"`).
- `event` (string): The current learner event. Supported values:
- `"INITIAL_SESSION"` – indicates the start of a new learning session.
- `"TOPIC_COMPLETE"` – indicates the learner has completed the active topic.
- `target_goal` (string): The learner's goal as provided by the user. Include this in **every** request.
- `context` (object): Session context. Must include:
- `session_id` (string): A unique session ID generated by your backend (e.g. a UUID). Use the *same* `session_id` for all calls in a session.
- On `"TOPIC_COMPLETE"`, also include `completed_topic` (string): the title of the topic just finished (exactly matching the previous `active_topic`).

Required fields: `learner_id`, `event`, `target_goal`, and `context.session_id`. For `"TOPIC_COMPLETE"`, also set `context.completed_topic`.

Example Requests

- **Initial Session (PowerShell):**

```
$body = @{
    learner_id   = "psychology-demo-001"
    event        = "INITIAL_SESSION"
    target_goal  = "I want to become a behavioural science researcher
specializing in how people make decisions and how cognitive biases
influence behaviour."
    context      = @{ session_id = "session-abc-123" }
} | ConvertTo-Json -Depth 5

$response = Invoke-RestMethod -Uri "http://127.0.0.1:8000/ai/
orchestrate" `
    -Method Post -ContentType "application/json" -Body $body

$response | ConvertTo-Json -Depth 10
```

- **Initial Session (cURL):**

```
curl -X POST http://127.0.0.1:8000/ai/orchestrate \
  -H "Content-Type: application/json" \
  -d '{
        "learner_id": "psychology-demo-001",
        "event": "INITIAL_SESSION",
        "target_goal": "I want to become a behavioural science
researcher specializing in how people make decisions and how cognitive
biases influence behaviour.",
        "context": { "session_id": "session-abc-123" }
      }'
```

- **Topic Complete (PowerShell):**

```
$body = @{
    learner_id   = "psychology-demo-001"
    event        = "TOPIC_COMPLETE"
    target_goal  = "I want to become a behavioural science researcher
specializing in how people make decisions and how cognitive biases
influence behaviour."
    context      = @{
        session_id       = "session-abc-123"
        completed_topic  = "Introduction to Psychology: Core Concepts"
    }
} | ConvertTo-Json -Depth 5

$response = Invoke-RestMethod -Uri "http://127.0.0.1:8000/ai/
```

```
orchestrate" `
    -Method Post -ContentType "application/json" -Body $body

$response | ConvertTo-Json -Depth 10
```

- **Topic Complete (cURL):**

```
curl -X POST http://127.0.0.1:8000/ai/orchestrate \
  -H "Content-Type: application/json" \
  -d '{
        "learner_id": "psychology-demo-001",
        "event": "TOPIC_COMPLETE",
        "target_goal": "I want to become a behavioural science
researcher specializing in how people make decisions and how cognitive
biases influence behaviour.",
        "context": {
          "session_id": "session-abc-123",
          "completed_topic": "Introduction to Psychology: Core Concepts"
        }
      }'
```

(These examples show how your Scala backend (or any HTTP client) should format the request. Simply replace the values with your session, learner, and topic data.)

Response Schema (OrchestrationResponse)

On success, the AI service returns **HTTP 200 OK** with a JSON body. The key fields are:

Field	Type	Description
session_id	string	The session ID (same as `context.session_id` in the request).
learner_id	string	Learner ID (echoes the request's `learner_id`).
status	string	Orchestration status. Usually `"READY"` when waiting for learner, or `"IN_PROGRESS"` if still computing.
active_topic	string	Title of the current active topic for the learner.
current_chunk	object	The current roadmap chunk: `{ sequence_number, topics (array), has_more (bool) }`.
available_topics	[string]	List of all topics in the current chunk (same as `current_chunk.topics`).

Field	Type	Description
active_resources	object	Resources found for the *active topic*. Contains:
- `youtube_resources`: array of resource objects (`{resource_id, title, url, ...}`),		
- `general_resources`: array of resource objects (`{resource_id, title, url, ...}`),		
- `summary`: a string summary of the resources.		
prefetched_topics	[string]	Titles of upcoming topics whose resources were proactively fetched.
can_continue	boolean	`true` if the session can proceed (indicates whether orchestration should continue).
more_roadmap_needed	boolean	`true` if more roadmap chunks may be needed in future.
next_recommended_action	string	The agent's suggested next action (e.g. `WAIT_FOR_LEARNER`, `PREPARE_TOPIC_RESOURCES`, etc.).
rationale	string	Explanation of the agent's decision or next step (for debugging/logging).

> **Resource objects:** Each item in `youtube_resources` or `general_resources` has at least `{ title, url }` (plus metadata like `resource_id`). The `url` field points to the video or resource link.

Example Response (INITIAL_SESSION)


```
{
  "session_id": "session-abc-123",
  "learner_id": "psychology-demo-001",
  "status": "READY",
  "active_topic": "Introduction to Psychology: Core Concepts",
  "current_chunk": {
    "sequence_number": 1,
    "topics": [
      "Introduction to Psychology: Core Concepts",
      "Cognitive Psychology: The Science of Thinking",
      "Statistics for Behavioural Sciences: Descriptive and Inferential"
    ],
    "has_more": true
  },
  "available_topics": [
```

```
    "Introduction to Psychology: Core Concepts",
    "Cognitive Psychology: The Science of Thinking",
    "Statistics for Behavioural Sciences: Descriptive and Inferential"
  ],
  "active_resources": {
    "youtube_resources": [
      {
        "resource_id": "yt123",
        "title": "Lecture: Core Concepts of Psychology",
        "url": "https://www.youtube.com/watch?v=abc123"
      }
    ],
    "general_resources": [
      {
        "resource_id": "res456",
        "title": "Intro to Psychology Article",
        "url": "https://example.com/intro-psychology"
      }
    ],
    "summary": "1 video and 1 article found for the topic."
  },
  "prefetched_topics": [
    "Cognitive Psychology: The Science of Thinking",
    "Statistics for Behavioural Sciences: Descriptive and Inferential"
  ],
  "can_continue": true,
  "more_roadmap_needed": false,
  "next_recommended_action": "WAIT_FOR_LEARNER",
  "rationale": "Active-topic resources prepared; next topics prefetched for
minimal latency."
}
```

> **Note:** The example above is a simplified illustration. Real responses will include more metadata in each resource object. The key points are: all URLs and required fields are present in `active_resources`, and upcoming topics appear in `prefetched_topics`.

Event Flow and Semantics

The orchestrator follows this event-driven flow:

1. **Initial Session (`INITIAL_SESSION`):**
2. The Scala backend calls `/ai/orchestrate` with event=`"INITIAL_SESSION"` and the learner's goal.
3. The AI service performs skill analysis and path planning, generating the first chunk of topics. It gathers resources for **Topic 1** and also prefetches resources for upcoming topics (Topic 2, Topic 3, etc.).
4. The response has `active_topic = Topic1`, the resources for Topic1 under `active_resources`, and `prefetched_topics = [Topic2, Topic3, ...]`.

5. The backend should display Topic1 and its resources to the learner.

6. **Learner Studies:**

7. The learner studies the active topic (Topic1). Meanwhile, resources for future topics (Topic2, Topic3) are already fetched and ready in the background (`prefetched_topics`).

8. **Topic Completion (`TOPIC_COMPLETE`):**

9. When the learner finishes Topic1, the Scala backend calls `/ai/orchestrate` with `event = "TOPIC_COMPLETE"` , the same `session_id` , and `context.completed_topic = "Topic1"` .

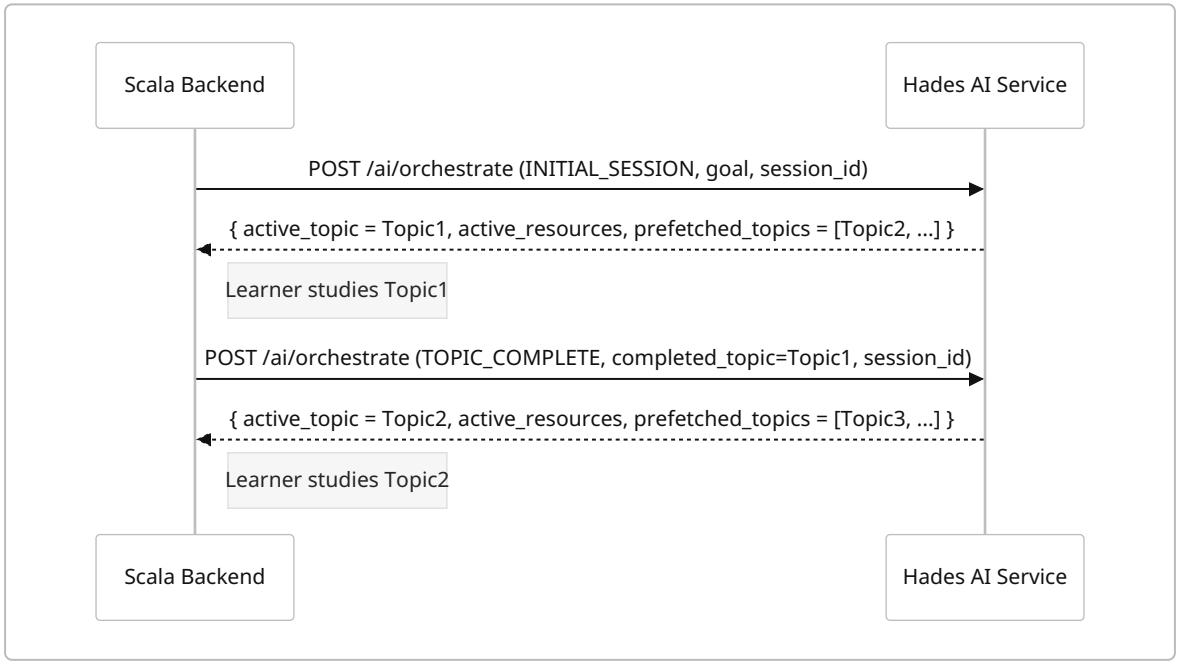
10. The AI marks Topic1 completed, makes Topic2 the active topic, ensures Topic2's resources are ready (from previous prefetch), and prefetches the next topic (Topic3) if not already done.

11. The response will have `active_topic = Topic2` , the resources for Topic2 in `active_resources` , and updated `prefetched_topics` (now starting with Topic3, etc.).

12. **Repeat:**

13. The backend updates its state and shows Topic2 to the learner. Repeat the process for each topic completion. Always use the same `session_id` to tie events together.

The sequence diagram below illustrates the interaction (Scala backend on left, AI Service on right):



Integration Responsibilities

- **Session Management:** The backend must generate and persist a unique `session_id` for each learning session. Include this ID in every request. **Do not** generate a new session ID when sending `TOPIC_COMPLETE` ; always use the same ID from the initial call.

- **State Persistence:** After each call, store the entire orchestration response (session ID, active topic, resources, etc.) in your database or memory. Treat the response as authoritative; **do not** attempt to recalc or override it in Scala.

- **Event Forwarding:** Map user actions to orchestrator events:

- On session start (first page load), send `INITIAL_SESSION`.

- After the learner indicates a topic is finished, send `TOPIC_COMPLETE` with the `completed_topic` matching the last `active_topic`.

- **Resource Delivery:** When the response includes `active_resources` (videos/articles for the current topic), make these URLs available to the frontend UI. The backend does **not** need to fetch or proxy these URLs – simply pass them through to the client.

- **Prefetched Topics:** The `prefetched_topics` list tells you which upcoming topics already have resources ready. The backend should store this list. You may use it to show in the UI that the next topics are preloaded. No additional API call is needed for these topics until the learner completes the current topic. (Always trust the AI: it is managing which topics are prefetched internally.)

- **Goal Consistency:** Always include the same `target_goal` string in each request (it can be identical to the initial one). This helps the AI maintain full context, even though it "knows" the goal from the first call.

- **No Hardcoding:** Do **not** encode any learning path or curriculum logic in the Scala code. For example, do not try to pick the "next topic" yourself. Always use the topic provided by the AI's `active_topic`. If the AI suggests `next_recommended_action = "WAIT_FOR_LEARNER"`, simply wait (show the topic to the user). If it suggests preparing resources, note that `active_resources` are already prepared by the AI. Trust the AI's instructions without second-guessing.

- **Configuration & Security:** Store your API keys and secrets (e.g. Mistral, Tavily) in the `.env` file only, and **never** commit them to source control【19†L265-L273】. Provide a `.env.example` with placeholder values so others can configure the service. If real keys are accidentally exposed (as they were here), rotate or revoke them immediately. Follow best practices for API key management (rotate keys periodically and if compromised)【19†L265-L273】.

Error Handling & Retry Guidance

- **HTTP Status:** The AI service is designed to always return **HTTP 200 OK** on a valid request, even if the orchestrator logic encounters a domain error. In case of a processing issue (validation error, no valid action, etc.), it will still return JSON (often with `status != "READY"` or an error message). Only treat actual HTTP errors (timeouts, 5xx responses) as communication failures. A non-200 status means a network or server error, not a business logic outcome.

- **Orchestration Errors:** If you see an `[ORCHESTRATOR] execution failure` or a traceback in the AI container's logs (as we did with `Connection pool acquisition timed out.`), the request likely had bad parameters. Common mistakes: mismatched `session_id`, missing `completed_topic`, or inconsistent fields. In such cases, log the response's `rationale` field

(it may contain a human-readable explanation) and double-check your request data. For example, use the container's logs to debug (e.g. look for the **[ORCHESTRATOR]** tag in Uvicorn logs).

- **Database Errors:** If the AI service can't reach the database (e.g. due to wrong `DATABASE_URL` or network), you'll see errors like *"could not translate host name"* or *"Connection refused"*. These indicate the container name/host or network is wrong【17†L1040-L1044】. Verify the DB container name, the Docker network, and check that Postgres is accepting connections (`pg_isready`). The TaskMaster guide notes that **use the container name, not** `localhost`, in your connection string【11†L108-L113】, and shows how a wrong host causes a failure.

- **Safety Ceiling:** The orchestrator imposes a reasoning step limit (default ~6 steps) to prevent runaway loops. If you see a log like `[ORCHESTRATOR] Safety ceiling reached`, it means the agent took too many nested reasoning steps. This should not happen in normal use. If it does, it might indicate a problem (like looping tasks). You can increase `max_steps` in the AI service config or examine your input logic if necessary.

- **Retries:** On transient failures (network timeouts, DNS hiccups, or 5xx errors), retry the request once or twice. The AI orchestration is idempotent for the same input. When retrying, send the identical JSON (same `session_id`, `event`, etc.) to avoid duplicating actions.

- **Validating Responses:** After each call, verify that `session_id` and `learner_id` in the response match your request. Also check key fields: for `INITIAL_SESSION`, ensure `active_topic` is non-empty; after a topic complete, ensure `active_resources` is present for the new topic. If critical fields are missing or empty, treat it as an error and retry or abort safely.

Testing Checklist

Before integrating into the Scala backend, manually test key scenarios:

1. **Start the Services:**

2. Start the PostgreSQL+pgvector container (see Startup steps above).

3. In the AI-service directory, ensure `.env` is correct (see `.env.example`).
4. Run: `docker run --rm -d --name hades-ai-service --network hades-net -p 8000:8000 --env-file ./.env hades-ai-service`.

5. Check the logs; you should see `Application startup complete. Uvicorn running on http://0.0.0.0:8000`.

6. **Initial Session Test:**

7. **PowerShell:**

```
$body = @{
    learner_id   = "psychology-demo-001"
    event        = "INITIAL_SESSION"
```

```
    target_goal  = "I want to become a behavioural science researcher
specializing in how people make decisions and how cognitive biases
influence behaviour."
    context      = @{ session_id = "session-1" }
} | ConvertTo-Json -Depth 5

$response = Invoke-RestMethod -Uri "http://127.0.0.1:8000/ai/
orchestrate" `
    -Method Post -ContentType "application/json" -Body $body
$response | ConvertTo-Json -Depth 10
```

8. **cURL:**

```
curl -X POST http://127.0.0.1:8000/ai/orchestrate \
  -H "Content-Type: application/json" \
  -d '{
        "learner_id": "psychology-demo-001",
        "event": "INITIAL_SESSION",
        "target_goal": "I want to become a behavioural science
researcher specializing in how people make decisions and how cognitive
biases influence behaviour.",
        "context": { "session_id": "session-1" }
      }'
```

9. **Verify:** HTTP **200 OK**. The JSON should include `active_topic`, `current_chunk`, `active_resources`, and `prefetched_topics`. There should be at least one URL in `active_resources.youtube_resources` and one in `active_resources.general_resources`. `status` should be `"READY"` (or `"PAUSED"` if still working). Check the AI service logs – you should see something like `POST /ai/orchestrate ... 200 OK` and **no** `[ORCHESTRATOR] execution failure` messages.

10. **Topic Completion Test:**

11. Suppose the initial response's `active_topic` was `"Introduction to Psychology: Core Concepts"`. Now simulate completing it.

12. **PowerShell:**

```
$body = @{
    learner_id   = "psychology-demo-001"
    event        = "TOPIC_COMPLETE"
    target_goal  = "I want to become a behavioural science researcher
specializing in how people make decisions and how cognitive biases
influence behaviour."
    context      = @{
        session_id       = "session-1"
        completed_topic  = "Introduction to Psychology: Core Concepts"
    }
} | ConvertTo-Json -Depth 5
```

```
$response = Invoke-RestMethod -Uri "http://127.0.0.1:8000/ai/
orchestrate" `
    -Method Post -ContentType "application/json" -Body $body
$response | ConvertTo-Json -Depth 10
```

13. **cURL:**

```
curl -X POST http://127.0.0.1:8000/ai/orchestrate \
  -H "Content-Type: application/json" \
  -d '{
        "learner_id": "psychology-demo-001",
        "event": "TOPIC_COMPLETE",
        "target_goal": "I want to become a behavioural science
researcher specializing in how people make decisions and how cognitive
biases influence behaviour.",
        "context": {
          "session_id": "session-1",
          "completed_topic": "Introduction to Psychology: Core Concepts"
        }
      }'
```

14. **Verify:** HTTP **200 OK**. The `active_topic` in the response should now be the next topic (e.g. `"Cognitive Psychology: The Science of Thinking"`). The `active_resources` should list URLs for this new topic. `prefetched_topics` should update accordingly (e.g. now starting at `"Statistics for Behavioural Sciences: ..."`, etc.). `status` should be `"READY"`. Logs should again show `POST /ai/orchestrate ... 200 OK` with no error traces.

15. **General Checks:**

16. After each request, ensure the AI logs show **no** errors (especially no `ORCHESTRATOR execution failure`).

17. Confirm `session_id` and `learner_id` in the response match your request.
18. Check that `next_recommended_action` makes sense (e.g. usually `WAIT_FOR_LEARNER` after preparing resources).
19. If anything looks wrong (fields missing, or error logs appear), double-check the request data, container networking, and database status.

Completing these steps will validate that the AI service and PostgreSQL database are correctly networked and that the `/ai/orchestrate` API is functioning as intended.

Citations: We followed Docker best practices to link separate containers (using a common network) 【4†L180-L184】【13†L232-L240】, and confirmed that the FastAPI app's connection string must use the Postgres container's name (not `localhost`)【11†L108-L113】. We also noted that `.env` should be kept out of version control【19†L265-L273】 and keys rotated if compromised. All guidance above aligns with these references.
